

```
function OBN_GUI1
% OBN 数据重定位、旋转和 HVSR 分析集成 GUI 平台

% 创建主窗口
fig = uifigure('Name', '海底节点 (OBN) 多分量地震数据预处理智能平台',
'Position', [100 100 900 700]);

% 创建选项卡组
tabgroup = uitabgroup(fig, 'Position', [20 20 860 660]);

%数据读取与预处理
tab1 = uitab(tabgroup, 'Title', '数据读取与预处理');
% 二次定位选项卡
tab2 = uitab(tabgroup, 'Title', '重定位');
% 旋转处理选项卡
tab3 = uitab(tabgroup, 'Title', '分量旋转');
% HVSR 分析选项卡
tab4 = uitab(tabgroup, 'Title', 'HVSR 分析');

%数据读取与预处理选项卡内容
% 文件选择区域
uipanel1 = uipanel(tab1, 'Title', '文件选择', 'Position', [20 450 820
150]);

% 第一行参数
uicontrol(uipanel1, 'Position', [20 98 100 22], 'Text', '文本头:');
textheader_dropdown = uidropdown(uipanel1,...
    'Items', {'否', '是'},...
    'Position', [130 98 80 22],...
    'Value', '是');

uicontrol(uipanel1, 'Position', [230 98 100 22], 'Text', '文本格式:');
textformat_dropdown = uidropdown(uipanel1,...
    'Items', {'自动检测', 'ascii', 'ebcdic'},...
    'Position', [340 98 100 22],...
    'Value', '自动检测');

uicontrol(uipanel1, 'Position', [460 98 100 22], 'Text', '浮点格式:');
fpformat_dropdown = uidropdown(uipanel1,...
    'Items', {'自动检测', 'ieee', 'ibm'},...
    'Position', [570 98 100 22],...
    'Value', '自动检测');

% 第二行参数
uicontrol(uipanel1, 'Position', [20 58 100 22], 'Text', '二进制头:');
binaryheader_dropdown = uidropdown(uipanel1,...
    'Items', {'否', '是'},...
    'Position', [130 58 80 22],...
    'Value', '是');

uicontrol(uipanel1, 'Position', [230 58 100 22], 'Text', '道头:');
traceheaders_dropdown = uidropdown(uipanel1,...
    'Items', {'否', '是'},...
    'Position', [340 58 80 22],...
    'Value', '是');
```

```
uicontrol(uipanel1, 'Position', [460 58 100 22], 'Text', '详细输出:');
verbose_dropdown = uicontrol(uipanel1,...
    'Items', {'否', '是'},...
    'Position', [570 58 80 22],...
    'Value', '是');

% 第三行参数
uicontrol(uipanel1, 'Position', [20 18 120 22], 'Text', '道范围 (如
1:100):');
traces_edit = uicontrol(uipanel1, 'text',...
    'Position', [130 18 80 22],...
    'Value', '',...
    'Placeholder', '留空读取全部道');

uicontrol(uipanel1, 'Position', [230 18 100 22], 'Text', 'SEGY 文件:');
seggy_file_edit = uicontrol(uipanel1, 'text', 'Position', [300 18 220
22],...
    'Value', '', 'Placeholder', '选择 SEGY 数据文件...');

browse_seggy_btn = uicontrol(uipanel1, 'push', 'Position', [570 18 100
22],...
    'Text', '浏览', 'ButtonPushedFcn', @browseSeggyFile);

% 读取参数区域
results_panel1 = uicontrol(tab1, 'Title', '读取参数', 'Position', [20 20 820
410]);

% 数据信息显示
info_text = uicontrol(results_panel1,...
    'Position', [20 150 500 140],...
    'Editable', 'off',...
    'Value', {'数据信息将显示在这里...'});

% 数据显示控制
uicontrol(results_panel1, 'Position', [20 110 100 22], 'Text', '显示道范
围:');
display_traces_edit = uicontrol(results_panel1, 'text',...
    'Position', [130 110 100 22],...
    'Value', '1:10:end',...
    'Placeholder', '如: 1:10:end');

display_btn = uicontrol(results_panel1, 'push',...
    'Position', [250 110 80 22],...
    'Text', '更新显示',...
    'ButtonPushedFcn', @updateDataDisplay);

% 创建坐标轴显示数据
data_axes = axes(results_panel1, 'Position', [550 50 350 240]);
title(data_axes, '地震数据显示');
xlabel(data_axes, '道号');
ylabel(data_axes, '采样点');

% 保存数据按钮
save_data_btn = uicontrol(results_panel1, 'push',...
    'Position', [20 50 120 30],...
    'Text', '保存数据',...
    'ButtonPushedFcn', @saveData);
```

```

        'ButtonPushedFcn', @saveSegyData,...
        'Enable', 'off');

%% 二次定位选项卡内容
% 参数输入区域
uipanel2 = uipanel(tab2, 'Title', '定位参数', 'Position', [20 500 820
100]);

uilabel(uipanel2, 'Position', [20 50 100 22], 'Text', '海水速度 (m/s):');
v0_edit = uieditfield(uipanel2, 'numeric', 'Position', [130 50 80 22],
'Value', 1500);

uilabel(uipanel2, 'Position', [220 50 120 22], 'Text', '时间漂移 (ms):');
tshift_edit = uieditfield(uipanel2, 'numeric', 'Position', [350 50 80 22],
'Value', 100);

uilabel(uipanel2, 'Position', [450 50 100 22], 'Text', '炮线深度 (m):');
srcLinez_edit = uieditfield(uipanel2, 'numeric', 'Position', [560 50 80
22], 'Value', 5);

uilabel(uipanel2, 'Position', [20 10 120 22], 'Text', '炮点抽稀系数:');
deltS_edit = uieditfield(uipanel2, 'numeric', 'Position', [130 10 80 22],
'Value', 20);

uilabel(uipanel2, 'Position', [220 10 120 22], 'Text', '参数文件:');
param_file_edit = uieditfield(uipanel2, 'text', 'Position', [350 10 200
22],...
'Value', 'OBS_par.csv');

browse_param_btn = uibutton(uipanel2, 'push', 'Position', [560 10 80
22],...
'Text', '浏览', 'ButtonPushedFcn', @browseParamFile);

% 执行按钮
run_loc_btn = uibutton(tab2, 'push', 'Position', [700 510 120 30],...
'Text', '执行重定位', 'ButtonPushedFcn', @runSecondaryLocation);

% 结果显示区域
results_panel2 = uipanel(tab2, 'Title', '定位结果', 'Position', [20 20 820
460]);

% 创建坐标轴显示定位结果
ax1 = uiaxes(results_panel2, 'Position', [50 50 700 300]);

%结果显示表格%
results_table = uitable(results_panel2, 'Position', [50 360 700 40],...
'ColumnName', {'OBS 编号', '原始 X', '原始 Y', '原始 Z', '调整 X', '调整
Y', '调整 Z', '速度'},...
'ColumnWidth', {60, 70, 70, 70, 70, 70, 70, 70});

%% 方位旋转选项卡内容
% 参数输入区域
uipanel3 = uipanel(tab3, 'Title', '旋转参数', 'Position', [20 500 820
100]);

uilabel(uipanel3, 'Position', [20 50 80 22], 'Text', 'OBS 编号:');
obs_id_edit = uieditfield(uipanel3, 'numeric', 'Position', [110 50 60 22],
'Value', 1);

```

```
uicontrol(uipanel3, 'Position', [190 50 100 22], 'Text', '时窗长度:');
winL_edit = uicontrol(uipanel3, 'numeric', 'Position', [300 50 60 22],
'Value', 20);

uicontrol(uipanel3, 'Position', [380 50 120 22], 'Text', '旋转参数文件:');
rotate_param_edit = uicontrol(uipanel3, 'text', 'Position', [510 50 200
22], ...
'Value', 'ObsRotate_par.csv');

browse_rotate_btn = uicontrol(uipanel3, 'push', 'Position', [720 50 80
22], ...
'Text', '浏览', 'ButtonPushedFcn', @browseRotateFile);

% 执行按钮
run_rotate_btn = uicontrol(tab3, 'push', 'Position', [700 510 120 30], ...
'Text', '执行分量旋转', 'ButtonPushedFcn', @runRotation);

% 结果显示区域
rotate_results_panel = uicontrol(tab3, 'Title', '旋转结果', 'Position', [20
20 820 460]);

% 创建三个坐标轴显示空间关系图
ax_orig = uiaxes(rotate_results_panel, 'Position', [50 220 220 200]);
title(ax_orig, '原始坐标系 (X-Y-Z)');
ax_rot = uiaxes(rotate_results_panel, 'Position', [300 220 220 200]);
title(ax_rot, '旋转后坐标系 (R-T-Z)');
ax_traj = uiaxes(rotate_results_panel, 'Position', [550 220 220 200]);
title(ax_traj, '水平旋转轨迹');

% 角度结果显示
uicontrol(rotate_results_panel, 'Position', [50 180 100 22], 'Text', '水
平旋转角度:');
theta_text = uicontrol(rotate_results_panel, 'Position', [150 180 80 22],
'Editable', false);

uicontrol(rotate_results_panel, 'Position', [250 180 100 22], 'Text', '
垂直旋转角度:');
thetaz_text = uicontrol(rotate_results_panel, 'Position', [350 180 80
22], 'Editable', false);

% 保存按钮
save_rotate_btn = uicontrol(rotate_results_panel, 'push', 'Position',
[500 180 100 22], ...
'Text', '保存旋转结果', 'ButtonPushedFcn', @saveRotateResults);

%% HVSR 分析选项卡内容
% 参数输入区域
uipanel4 = uicontrol(tab4, 'Title', 'HVSR 分析参数', 'Position', [20 500 820
100]);

uicontrol(uipanel4, 'Position', [20 50 100 22], 'Text', '采样间隔 (s):');
dt_edit = uicontrol(uipanel4, 'numeric', 'Position', [130 50 60 22],
'Value', 0.01);

uicontrol(uipanel4, 'Position', [210 50 100 22], 'Text', '平滑窗口:');
window_edit = uicontrol(uipanel4, 'numeric', 'Position', [320 50 60 22],
'Value', 5);
```

```
% 执行按钮
run_hvsr_btn = uibutton(tab4, 'push', 'Position', [700 510 120 30],...
    'Text', '执行 HVSR 分析', 'ButtonPushedFcn', @runHVSRAnalysis);

% 结果显示区域
hvsr_results_panel = uipanel(tab4, 'Title', 'HVSR 分析结果', 'Position',
    [20 20 820 460]);

% 创建坐标轴显示 HVSR 曲线
ax4 = uiaxes(hvsr_results_panel, 'Position', [50 50 700 350]);

% 峰值信息显示
uilabel(hvsr_results_panel, 'Position', [50 410 150 22], 'Text', '主要
峰值频率:');
peak_freq_text = uitable(hvsr_results_panel, 'Position', [200 410 100
22], 'Editable', false);

uilabel(hvsr_results_panel, 'Position', [320 410 150 22], 'Text', '峰
值 HVSR 值:');
peak_value_text = uitable(hvsr_results_panel, 'Position', [470 410
100 22], 'Editable', false);

% 保存按钮
save_hvsr_btn = uibutton(hvsr_results_panel, 'push', 'Position', [600
410 120 22],...
    'Text', '保存 HVSR 数据', 'ButtonPushedFcn', @saveHVSROutput);

%% 回调函数
function browseParamFile(src, event)
    [file, path] = uigetfile('*.csv', '选择参数文件');
    if file ~= 0
        param_file_edit.Value = fullfile(path, file);
    end
end

function browseRotateFile(src, event)
    [file, path] = uigetfile('*.csv', '选择旋转参数文件');
    if file ~= 0
        rotate_param_edit.Value = fullfile(path, file);
    end
end

function runSecondaryLocation(src, event)
    try
        % 获取参数
        v0 = v0_edit.Value;
        tshift = tshift_edit.Value;
        srcLinez = srcLinez_edit.Value;

        % 执行二次定位
        [OBS, SRCLine] = MainSecondaryLocation(v0, tshift, srcLinez);

        % 在坐标轴上显示结果
        cla(ax1);
        plot(ax1, SRCLine.x, SRCLine.y, 'k.', 'MarkerSize', 1);
        hold(ax1, 'on');
```

```

% 更新表格数据
tableData = [];
for k = 1:length(OBS)
    plot(ax1, OBS{k}.xOrg, OBS{k}.yOrg, 'bo',
'MarkerFaceColor', 'b');
    plot(ax1, OBS{k}.xAdj, OBS{k}.yAdj, 'ro',
'MarkerFaceColor', 'r');

    text(ax1, OBS{k}.xOrg+20, OBS{k}.yOrg, num2str(k));
    text(ax1, OBS{k}.xAdj+20, OBS{k}.yAdj, ...
        sprintf('δx=%.1f', OBS{k}.xOrg-OBS{k}.xAdj));
    text(ax1, OBS{k}.xAdj+20, OBS{k}.yAdj+100, ...
        sprintf('δy=%.1f', OBS{k}.yOrg-OBS{k}.yAdj));
    text(ax1, OBS{k}.xAdj+20, OBS{k}.yAdj+200, ...
        sprintf('δz=%.1f', OBS{k}.zOrg-OBS{k}.zAdj));

    tableData = [tableData; {k, OBS{k}.xOrg, OBS{k}.yOrg,
OBS{k}.zOrg, ...
        OBS{k}.xAdj, OBS{k}.yAdj, OBS{k}.zAdj, OBS{k}.v0Adj}];
end

results_table.Data = tableData;
axis(ax1, 'ij');
xlabel(ax1, 'X 坐标');
ylabel(ax1, 'Y 坐标');
title(ax1, '二次定位结果（蓝色:原始位置，红色:调整后位置）');
legend(ax1, '炮点', '原始OBS', '调整后OBS', 'Location', 'best');
grid(ax1, 'on');

% 保存结果
save('OBSLocationAdj.mat', 'OBS');
updateStatus('二次定位完成! 结果已保存到 OBSLocationAdj.mat');

catch ME
    errordlg(sprintf('执行二次定位时出错: %s', ME.message), '错误');
    updateStatus('二次定位执行失败!');
end
end

function runRotation(src, event)
try
    % 检查是否已进行二次定位
    if ~exist('OBSLocationAdj.mat', 'file')
        errordlg('请先执行二次定位!', '错误');
        updateStatus('错误: 请先执行二次定位!');
        return;
    end

    % 加载二次定位结果
    load('OBSLocationAdj.mat', 'OBS');

    % 获取参数
    obs_id = obs_id_edit.Value;
    winL = winL_edit.Value;

    % 执行旋转（使用修改版的函数）
    rot_results = MainRotate_GUI(OBS, obs_id, winL);

    % 显示旋转角度

```

```

theta_text.Value = sprintf('%.1f°', rot_results.theta);
thetaz_text.Value = sprintf('%.1f°', rot_results.thetaz);

% 在旋转结果面板中显示空间关系图
showSpatialRotationInPanel(rot_results.theta,
rot_results.thetaz, ax_orig, ax_rot, ax_traj);

updateStatus('方位旋转处理完成!');

catch ME
    errordlg(sprintf('执行方位旋转时出错: %s', ME.message), '错误');
    updateStatus('方位旋转执行失败!');
end
end

function saveRotateResults(src, event)
    try
        if exist('ObsRotate_par.mat', 'file')
            updateStatus('旋转结果已保存到 ObsRotate_par.mat');
        else
            errordlg('未找到旋转结果文件!', '错误');
        end
    catch ME
        errordlg(sprintf('保存旋转结果时出错: %s', ME.message), '错误');
    end
end

function runHVSRAnalysis(src, event)
    try
        % 检查是否已进行方位旋转
        if ~exist('ObsRotate_par.mat', 'file')
            errordlg('请先执行分量旋转!', '错误');
            updateStatus('错误: 请先执行分量旋转!');
            return;
        end

        % 获取参数
        dt = dt_edit.Value;
        window_size = window_edit.Value;

        % 执行 HVSR 分析
        [f, HVSR, HVSR_smoothed, peak_freq, peak_value] =
GenerateHVSR_GUI(dt, window_size);

        % 在坐标轴上显示 HVSR 曲线
        cla(ax4);
        semilogx(ax4, f, HVSR, 'b-', 'LineWidth', 1);
        hold(ax4, 'on');
        semilogx(ax4, f, HVSR_smoothed, 'r-', 'LineWidth', 2);
        xlabel(ax4, 'Frequency (Hz)');
        ylabel(ax4, 'HVSR (dB)');
        title(ax4, 'Horizontal-to-Vertical Spectral Ratio (HVSR)');
        grid(ax4, 'on');
        legend(ax4, '原始', '平滑后', 'Location', 'best');

        % 显示峰值信息
        peak_freq_text.Value = sprintf('%.3f Hz', peak_freq);
        peak_value_text.Value = sprintf('%.2f dB', peak_value);
    end
end

```

```

        updateStatus('HVSr 分析完成! ');

        catch ME
            errordlg(sprintf('执行 HVSr 分析时出错: %s', ME.message), '错误
        ');
        updateStatus('HVSr 分析执行失败! ');
    end
end

function saveHVSrOutput(src, event)
    try
        if exist('HVSr_Data.txt', 'file')
            updateStatus('HVSr 数据已保存到 HVSr_Data.txt');
        else
            errordlg('未找到 HVSr 数据文件!', '错误');
        end
    catch ME
        errordlg(sprintf('保存 HVSr 数据时出错: %s', ME.message), '错误
    ');
    end
end

function updateStatus(message)
    status_label.Text = [datestr(now, 'HH:MM:SS') ' - ' message];
    drawnow;
end

% 状态栏
status_label = uilabel(fig, 'Position', [20 5 860 15],...
    'Text', '就绪 - 请选择处理功能', 'FontColor', 'blue');

end

%% 空间关系图显示函数（嵌入面板版本）
function showSpatialRotationInPanel(theta, thetaz, ax_orig, ax_rot,
ax_traj)
    % 在给定的坐标轴中显示空间旋转关系

    % 子图 1: 原始坐标系
    cla(ax_orig);
    % 显示原始坐标系和分量
    quiver3(ax_orig, 0,0,0,1,0,0,'r','LineWidth',2,'MaxHeadSize',0.3);
    hold(ax_orig, 'on');
    quiver3(ax_orig, 0,0,0,0,1,0,'g','LineWidth',2,'MaxHeadSize',0.3);
    quiver3(ax_orig, 0,0,0,0,0,1,'b','LineWidth',2,'MaxHeadSize',0.3);

    % 添加标签
    text(ax_orig,
    1.1,0,0,'X','FontSize',10,'FontWeight','bold','Color','r');
    text(ax_orig,
    0,1.1,0,'Y','FontSize',10,'FontWeight','bold','Color','g');
    text(ax_orig,
    0,0,1.1,'Z','FontSize',10,'FontWeight','bold','Color','b');

    title(ax_orig, '原始坐标系 (X-Y-Z)');
    grid(ax_orig, 'on'); axis(ax_orig, 'equal');
    xlabel(ax_orig, 'X 轴'); ylabel(ax_orig, 'Y 轴'); zlabel(ax_orig, 'Z
    轴');
    xlim(ax_orig, [-1.5 1.5]); ylim(ax_orig, [-1.5 1.5]); zlim(ax_orig,

```

```

[-1.5 1.5]);
view(ax_orig, 45,30);

% 子图 2: 旋转后的坐标系
cla(ax_rot);
% 计算旋转矩阵
R = rotationMatrix(theta, thetaz);

% 显示旋转后的坐标系
x_rot = R * [1;0;0];
y_rot = R * [0;1;0];
z_rot = R * [0;0;1];

quiver3(ax_rot,
0,0,0,x_rot(1),x_rot(2),x_rot(3),'r','LineWidth',2,'MaxHeadSize',0.3);
hold(ax_rot, 'on');
quiver3(ax_rot,
0,0,0,y_rot(1),y_rot(2),y_rot(3),'g','LineWidth',2,'MaxHeadSize',0.3)
;
quiver3(ax_rot,
0,0,0,z_rot(1),z_rot(2),z_rot(3),'b','LineWidth',2,'MaxHeadSize',0.3)
;

% 添加标签
text(ax_rot, x_rot(1)*1.1, x_rot(2)*1.1, x_rot(3)*1.1,
'R','FontSize',10,'FontWeight','bold','Color','r');
text(ax_rot, y_rot(1)*1.1, y_rot(2)*1.1, y_rot(3)*1.1,
'T','FontSize',10,'FontWeight','bold','Color','g');
text(ax_rot, z_rot(1)*1.1, z_rot(2)*1.1, z_rot(3)*1.1,
'Z','FontSize',10,'FontWeight','bold','Color','b');

title(ax_rot, sprintf('旋转后坐标系 \n $\theta= %.1f^\circ$ ,  $\theta_z= %.1f^\circ$ ', theta,
thetaz));
grid(ax_rot, 'on'); axis(ax_rot, 'equal');
xlabel(ax_rot, 'X 轴'); ylabel(ax_rot, 'Y 轴'); zlabel(ax_rot, 'Z 轴
');
xlim(ax_rot, [-1.5 1.5]); ylim(ax_rot, [-1.5 1.5]); zlim(ax_rot,
[-1.5 1.5]);
view(ax_rot, 45,30);

% 子图 3: 旋转过程示意图
cla(ax_traj);
% 显示旋转轨迹
theta_rad = deg2rad(theta);
t = linspace(0, theta_rad, 50);

% 计算旋转轨迹
x_traj = cos(t);
y_traj = sin(t);

plot(ax_traj, [0 1], [0 0], 'k--', 'LineWidth', 1); hold(ax_traj,
'on');
plot(ax_traj, x_traj, y_traj, 'm-', 'LineWidth', 2);
plot(ax_traj, [0 cos(theta_rad)], [0 sin(theta_rad)], 'r-',
'LineWidth', 2);

% 标记角度
text(ax_traj, 0.3, 0.1, sprintf(' $\theta= %.1f^\circ$ ', theta), 'FontSize', 10);

```

```
% 添加单位圆
theta_circle = linspace(0, 2*pi, 100);
plot(ax_traj, cos(theta_circle), sin(theta_circle), 'k:',
'LineWidth', 0.5);

title(ax_traj, '水平旋转轨迹');
xlabel(ax_traj, 'X 轴'); ylabel(ax_traj, 'Y 轴');
grid(ax_traj, 'on'); axis(ax_traj, 'equal');
xlim(ax_traj, [-1.2 1.2]); ylim(ax_traj, [-1.2 1.2]);
end

%% 旋转矩阵计算函数
function R = rotationMatrix(theta, thetaz)
% 计算旋转矩阵
% theta: 水平旋转角度 (度)
% thetaz: 垂直旋转角度 (度)

theta_rad = deg2rad(theta);
thetaz_rad = deg2rad(thetaz);

% 水平旋转矩阵 (绕 z 轴)
Rz = [cos(theta_rad), -sin(theta_rad), 0;
      sin(theta_rad), cos(theta_rad), 0;
      0, 0, 1];

% 垂直旋转矩阵 (绕 x 轴)
Rx = [1, 0, 0;
      0, cos(thetaz_rad), -sin(thetaz_rad);
      0, sin(thetaz_rad), cos(thetaz_rad)];

% 组合旋转: 先水平后垂直
R = Rx * Rz;
end

%% 修改版的 MainRotate 函数 (适配 GUI)
function results = MainRotate_GUI(OBS, obs_id, winL)
% 基于 MainRotate 修改的 GUI 版本, 返回旋转结果

% 设置参数
xgeophone = OBS{obs_id}.xAdj;
ygeophone = OBS{obs_id}.yAdj;
zgeophone = OBS{obs_id}.zAdj;
srcLinez = 0; % 炮线深度

% 读取参数文件
[numericData, textData, rawData] = xlsread("ObsRotate_par.csv");

% 文件路径
fpath = char(textData(1));
fname_x = char(textData(2));
fname_y = char(textData(3));
fname_z = char(textData(4));

% 加载原始数据
Fname_x = fullfile(fpath, fname_x);
Fname_y = fullfile(fpath, fname_y);
Fname_z = fullfile(fpath, fname_z);
```

```
[fname, file_headX, traceX_head, traceX, dt] = readsgyIBM(Fname_x);
[fname, file_headY, traceY_head, traceY, dt] = readsgyIBM(Fname_y);
[fname, file_headZ, traceZ_head, traceZ, dt] = readsgyIBM(Fname_z);

% 执行旋转处理（这里简化显示，实际应执行完整的旋转算法）
% 注意：这里需要调用实际的旋转算法，以下为示例

% 模拟旋转结果
theta = 45.5; % 示例角度
thetaz = -12.3; % 示例角度

% 返回结果
results.theta = theta;
results.thetaz = thetaz;
results.traceX = traceX;
results.traceY = traceY;
results.traceZ = traceZ;

% 保存结果
save('ObsRotate_par.mat', 'traceX', 'traceY', 'traceZ', '-append');
end

%% 修改后的 HVSR 函数（适配 GUI）
function [f, HVSR, HVSR_smoothed, peak_freq, peak_value] =
GenerateHVSR_GUI(dt, window_size)
% 修改版的 HVSR 函数，适配 GUI 调用

% 步骤 1：加载旋转后的 OBN 数据
load('ObsRotate_par.mat'); % 加载旋转后的数据

% 步骤 2：计算频谱
Fs = 1/dt; % 采样率
nfft = 2^nextpow2(size(traceX, 1)); % FFT 点数

% 计算水平分量频谱（取几何平均）
H1_spectrum = mean(abs(fft(traceX, nfft, 1)), 2); % 通道平均
H2_spectrum = mean(abs(fft(traceY, nfft, 1)), 2); % 若存在 Y 分量
H_avg = sqrt(H1_spectrum .* H2_spectrum); % 水平几何平均

% 计算垂直分量频谱
V_spectrum = mean(abs(fft(traceZ, nfft, 1)), 2); % 垂直分量

% 频率向量
f = Fs/2 * linspace(0, 1, nfft/2+1);

% 步骤 3：计算 HVSR
HVSR_linear = H_avg(1:nfft/2+1) ./ V_spectrum(1:nfft/2+1);
HVSR = 10*log10(HVSR_linear); % 转换为 dB

% 步骤 4：平滑处理
HVSR_smoothed = movmean(HVSR, window_size);

% 步骤 5：寻找主要峰值
[peak_value, peak_idx] = max(HVSR_smoothed);
peak_freq = f(peak_idx);

% 步骤 6：保存为文本文件
```

```
hvsr_data = [f(:), HVSR(:), HVSR_smoothed(:)];  
header = 'Frequency(Hz)\tHVSR(dB)\tHVSR_Smoothed(dB)';  
fid = fopen('HVSR_Data.txt', 'w');  
fprintf(fid, '%s\n', header);  
fprintf(fid, '%.4f\t%.4f\t%.4f\n', hvsr_data);  
fclose(fid);  
end
```